

MS571/MT571 - Aprendizado de Máquinas: Aspectos Teóricos e Práticos

Projeto 01: Reconhecimento de Dígitos Manuscritos

Gustavo Oliveira Nunes - RA 244981
João Luís Motta Martinez - RA 253165
Luisa de Melo Carneiro - RA 239716
Maria Eduarda Villéla Silva - RA 212974

Campinas - SP
Setembro de 2026

Sumário

1	Introdução	3
2	Metodologia	4
2.1	Pré-processamento e representação dos dados	4
2.2	Arquitetura da rede neural	4
2.2.1	Ativação Sigmoidal	4
2.3	Forward Propagation e custo	5
2.4	Backpropagation	5
2.5	Inicialização Aleatória de Pesos	6
2.6	Algoritmos de Otimização	6
2.7	Checação numérica do gradiente	6
2.8	Estratégia de seleção de modelo e divisão dos dados	6
3	Resultados e Discussão	8
3.1	Amostra da base e treinamento inicial por gradiente descendente	8
3.2	Validação da checagem do gradiente	9
3.3	Comparação de algoritmos de otimização (Parte I)	9
3.4	Visualização das unidades escondidas	10
3.5	Curva de aprendizado (Parte II)	11
3.6	Seleção automática do parâmetro de regularização	12
3.7	Avaliação final no conjunto de teste	13
4	Conclusão	15

1 Introdução

Este relatório apresenta o desenvolvimento, os testes e a avaliação crítica de uma rede neural regularizada com uma camada escondida para a base MNIST, bem como a implementação de uma rede neural *feedforward* totalmente conectada, com uma camada escondida, treinada por meio do algoritmo de *backpropagation* e regularizada para controlar o sobreajuste (*overfitting*).

O trabalho está dividido em duas partes:

- **Parte I – Redes Neurais:** implementação da arquitetura, da função de custo regularizada, do *backpropagation*, da checagem numérica do gradiente, do treinamento via gradiente conjugado e da visualização das unidades escondidas.
- **Parte II – Seleção de Modelo:** divisão dos dados em treino, validação e teste; cálculo da curva de aprendizado; e automatização da escolha do hiperparâmetro de regularização λ com base no conjunto de validação.

Foi garantido que o código funcionasse de forma genérica para qualquer número de exemplos de treinamento, de classes, de camadas escondidas e de neurônios por camada.

2 Metodologia

2.1 Pré-processamento e representação dos dados

A base de dados é fornecida em dois arquivos CSV: `imageMNIST.csv`, contendo 5 000 imagens de dimensão original 20×20 pixels vetorizadas em linhas de 400 elementos ($\mathbf{x}^{(i)} \in \mathbb{R}^{400}$), e `labelMNIST.csv`, contendo os rótulos correspondentes $y^{(i)} \in \{1, 2, \dots, 10\}$. Pela convenção adotada, o rótulo 10 foi convertido para a classe 0, resultando em $y \in \{0, 1, \dots, 9\}$.

Para o treinamento com função de custo de entropia cruzada multiclasse, os rótulos escalares foram convertidos para a representação *one-hot encoding*. Cada rótulo $y^{(i)}$ é codificado em um vetor binário $Y^{(i)} \in \mathbb{R}^{10}$, onde a posição correspondente à classe verdadeira recebe valor 1 e todas as demais recebem 0:

$$Y_{i,k} = \begin{cases} 1, & \text{se } y^{(i)} = k \\ 0, & \text{caso contrário} \end{cases} \quad \text{para } k \in \{0, 1, \dots, 9\} \quad (1)$$

2.2 Arquitetura da rede neural

A arquitetura utilizada é um Perceptron Multicamadas composto por 3 camadas ($L = 3$):

1. **Camada 1 (Entrada):** $s_1 = 400$ unidades de entrada (correspondentes aos 400 pixels da imagem) mais uma unidade de *bias* $a_0^{(1)} = 1$.
2. **Camada 2 (Escondida):** $s_2 = 25$ unidades de ativação mais uma unidade de *bias* $a_0^{(2)} = 1$.
3. **Camada 3 (Saída):** $s_3 = 10$ unidades de saída (uma para cada dígito).

O código foi implementado de forma genérica, representando a arquitetura por uma lista de tamanhos de camada (por exemplo, `[400, 25, 10]`), o que permite treinar redes com qualquer número de camadas escondidas e de unidades por camada sem alterar as rotinas de custo ou de *backpropagation*.

2.2.1 Ativação Sigmoidal

A função de ativação utilizada em todas as unidades das camadas escondida e de saída é a função logística (sigmoide):

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2)$$

Sua derivada em relação à entrada é expressa diretamente em termos de sua saída:

$$\sigma'(z) = \sigma(z) (1 - \sigma(z)) \quad (3)$$

Para evitar *overflow* decorrente de valores extremamente negativos de z durante as iterações de otimização (onde e^{-z} pode exceder a capacidade de representação em ponto flutuante), implementou-se uma versão numericamente estável:

$$\sigma(z) = \begin{cases} \frac{1}{1 + e^{-z}}, & \text{se } z \geq 0 \\ \frac{e^z}{1 + e^z}, & \text{se } z < 0 \end{cases} \quad (4)$$

2.3 Forward Propagation e custo

Dada a matriz de dados de entrada $X \in \mathbb{R}^{m \times 400}$, adiciona-se a coluna de *bias* de uns, resultando em $A^{(1)} \in \mathbb{R}^{m \times 401}$. As combinações lineares $Z^{(l)}$ e ativações $A^{(l)}$ para cada camada l são calculadas por multiplicações matriciais vetorizadas:

$$Z^{(2)} = A^{(1)}(\Theta^{(1)})^T \in \mathbb{R}^{m \times 25} \quad (5)$$

$$A^{(2)} = \begin{bmatrix} \mathbf{1}_{m \times 1} & \sigma(Z^{(2)}) \end{bmatrix} \in \mathbb{R}^{m \times 26} \quad (6)$$

$$Z^{(3)} = A^{(2)}(\Theta^{(2)})^T \in \mathbb{R}^{m \times 10} \quad (7)$$

$$h_{\Theta}(X) = A^{(3)} = \sigma(Z^{(3)}) \in \mathbb{R}^{m \times 10} \quad (8)$$

A função de custo $J(\Theta)$ para a rede neural multiclasse com regularização L_2 (Ridge) é dada por:

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K \left[Y_{i,k} \ln \left(h_{\Theta}(x^{(i)})_k \right) + (1 - Y_{i,k}) \ln \left(1 - h_{\Theta}(x^{(i)})_k \right) \right] + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_{l+1}} \sum_{j=1}^{s_l} \left(\Theta_{i,j}^{(l)} \right)^2 \quad (9)$$

Os parâmetros de *bias* ($\Theta_{i,0}^{(l)}$) não são penalizados na soma de regularização. Para assegurar estabilidade numérica no cálculo do logaritmo da entropia cruzada, utilizou-se a formulação equivalente via $Z^{(3)}$ (*softplus trick*):

$$\text{loss}(Z, Y) = \max(Z, 0) - Y \cdot Z + \ln \left(1 + e^{-|Z|} \right) \quad (10)$$

que evita o cálculo direto de exponenciais de números muito grandes, problema comum quando a sigmoide satura próximo de 0 ou 1.

2.4 Backpropagation

O algoritmo de *backpropagation* calcula as derivadas parciais $\partial J / \partial \Theta_{ij}^{(l)}$ de forma eficiente através da regra da cadeia. O procedimento vetorizado para todo o conjunto de treinamento é:

1. Erro na camada de saída ($L = 3$):

$$\delta^{(3)} = A^{(3)} - Y \in \mathbb{R}^{m \times 10} \quad (11)$$

2. Propagação do erro para a camada escondida ($L = 2$):

$$\delta^{(2)} = \left(\delta^{(3)} \Theta_{[:,1:]}^{(2)} \right) \circ \sigma'(Z^{(2)}) \in \mathbb{R}^{m \times 25} \quad (12)$$

onde $\circ$ denota o produto Hadamard (ponto a ponto) e $\Theta_{[:,1:]}^{(2)}$ ignora a coluna de *bias*.

3. Acúmulo dos gradientes:

$$\Delta^{(1)} = (\delta^{(2)})^T A^{(1)} \in \mathbb{R}^{25 \times 401}, \quad \Delta^{(2)} = (\delta^{(3)})^T A^{(2)} \in \mathbb{R}^{10 \times 26} \quad (13)$$

4. Gradiente final regularizado:

$$D^{(l)} = \frac{\partial J}{\partial \Theta^{(l)}} = \frac{1}{m} \Delta^{(l)} + \frac{\lambda}{m} \tilde{\Theta}^{(l)}, \quad \tilde{\Theta}^{(l)} = \begin{bmatrix} \mathbf{0}_{s_{l+1} \times 1} & \Theta_{[:,1:]}^{(l)} \end{bmatrix} \quad (14)$$

2.5 Inicialização Aleatória de Pesos

A inicialização de todos os pesos com zero (ou valores constantes) levaria à simetria indesejada: todas as unidades escondidas calculariam exatamente as mesmas ativações e receberiam os mesmos gradientes, reduzindo a camada escondida a um único neurônio efetivo. Para quebrar essa simetria, os pesos foram inicializados aleatoriamente a partir de uma distribuição uniforme no intervalo $[-\epsilon, \epsilon]$, com

$$\epsilon = \frac{\sqrt{6}}{\sqrt{L_{\text{in}} + L_{\text{out}}}}. \quad (15)$$

Para a primeira camada ($L_{\text{in}} = 400$, $L_{\text{out}} = 25$), obtém-se $\epsilon \approx 0,1189$.

2.6 Algoritmos de Otimização

Foram comparados dois métodos de otimização para minimizar $J(\Theta)$:

1. **Gradiente Descendente (*Batch Gradient Descent*)**: atualização direta com taxa de aprendizado fixa α : $\Theta^{(l)} := \Theta^{(l)} - \alpha D^{(l)}$.
2. **Gradiente Conjugado (CG)**: otimizador N -dimensional baseado em direções conjugadas, via a rotina `scipy.optimize.minimize (method='CG')`. O otimizador recebe uma função que retorna simultaneamente o custo $J(\Theta)$ e o vetor desenrolado de gradientes $\nabla J(\Theta)$, calculados pelo *backpropagation* implementado neste trabalho; apenas a busca da direção e do passo de atualização é delegada à biblioteca.

2.7 Checagem numérica do gradiente

Para validar a correção da implementação do *backpropagation*, implementou-se o algoritmo de checagem do gradiente por diferenças finitas centrais. Devido ao alto custo computacional do cálculo numérico, o teste foi realizado em uma rede miniatura com os parâmetros recomendados no enunciado: $m = 3$ exemplos aleatórios, $L_{\text{entrada}} = 3$, $L_{\text{escondida}} = 5$, $L_{\text{saída}} = 3$, com perturbação $\epsilon = 10^{-4}$. A aproximação numérica da i -ésima componente do gradiente é

$$g_{\text{num},i} = \frac{J(\theta_i + \epsilon) - J(\theta_i - \epsilon)}{2\epsilon}, \quad (16)$$

e a diferença relativa entre o gradiente analítico (g_{backprop}) e o numérico (g_{num}) é

$$\text{Diferença Relativa} = \frac{\|g_{\text{num}} - g_{\text{backprop}}\|_2}{\max(\|g_{\text{num}} + g_{\text{backprop}}\|_2, 10^{-15})}. \quad (17)$$

Valores muito inferiores a 10^{-7} confirmam a ausência de erros na implementação analítica do gradiente.

2.8 Estratégia de seleção de modelo e divisão dos dados

Para avaliar a capacidade de generalização sem vazamento de dados (*data leakage*), o conjunto original de 5 000 amostras foi dividido em três subconjuntos independentes por amostragem estratificada, preservando a proporção das 10 classes em cada subconjunto: treino (60%, 3 000 imagens), validação (20%, 1 000 imagens, usada para a curva de aprendizado e para escolher λ^*) e teste (20%, 1 000 imagens, mantido isolado até a avaliação final).

Tabela 1: Distribuição estratificada das amostras por classe nos conjuntos de dados.

Classe	Treino (60%)	Validação (20%)	Teste (20%)	Total
0–9 (cada)	300	100	100	500
Total	3 000	1 000	1 000	5 000

A curva de aprendizado foi gerada treinando subconjuntos crescentes do conjunto de treino (10 tamanhos, de 100 até 3 000 exemplos) com $\lambda = 1$ fixo, avaliando o custo sem regularização no subconjunto de treino e em todo o conjunto de validação. A busca automatizada do valor ótimo de λ avaliou os 10 candidatos sugeridos no enunciado, $\lambda \in \{0; 0,001; 0,003; 0,01; 0,03; 0,1; 0,3; 1; 3; 10\}$, treinando a rede apenas com o conjunto de treino para cada candidato e escolhendo λ^* como o valor que minimizou o custo de validação sem regularização. É importante destacar que, em toda a Parte II, o conjunto de teste foi mantido rigorosamente fora tanto do treinamento da rede quanto do processo de escolha de λ : nem a curva de aprendizado, nem a busca automatizada de λ tiveram qualquer acesso às amostras de teste, que permaneceram isoladas até a etapa final de avaliação (Seção 3.7). Esse cuidado metodológico evita o vazamento de dados (*data leakage*) e garante que a acurácia final reportada no conjunto de teste reflita, de fato, a capacidade de generalização do modelo para dados nunca vistos durante o ajuste de parâmetros ou de hiperparâmetros, e não uma estimativa viesada pelo próprio processo de seleção do modelo.

3 Resultados e Discussão

3.1 Amostra da base e treinamento inicial por gradiente descendente

A Figura 1 mostra uma amostra de imagens da base utilizada. Com a arquitetura $[400, 25, 10]$, $\lambda = 1$, $\alpha = 1$ e 500 iterações de gradiente descendente sobre os 5 000 exemplos, a rede atingiu **93,74%** de acurácia no conjunto de treinamento. A Figura 2 mostra a evolução do custo regularizado.



Figura 1: Amostra aleatória de imagens 28×28 da base de dados.

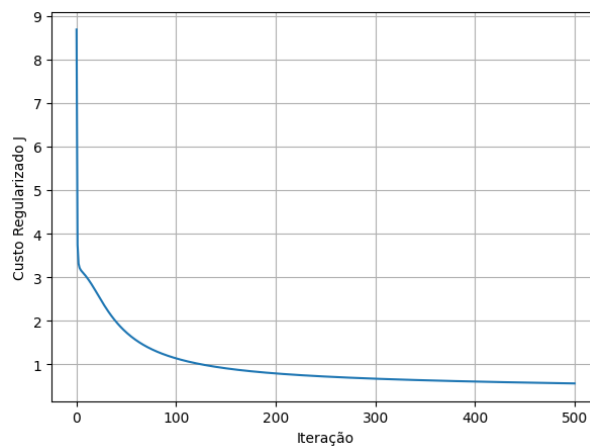


Figura 2: Evolução do custo regularizado $J(\Theta)$ durante o treinamento por gradiente descendente.

3.2 Validação da checagem do gradiente

A checagem numérica foi executada na rede miniatura ($3 \rightarrow 5 \rightarrow 3$, $m = 3$), sem e com regularização. A Tabela 2 e a Figura 3 mostram os resultados.

Tabela 2: Diferença relativa entre gradiente numérico e gradiente de backpropagation

λ	Diferença relativa
0	$1,11 \times 10^{-11}$
1	$1,14 \times 10^{-11}$

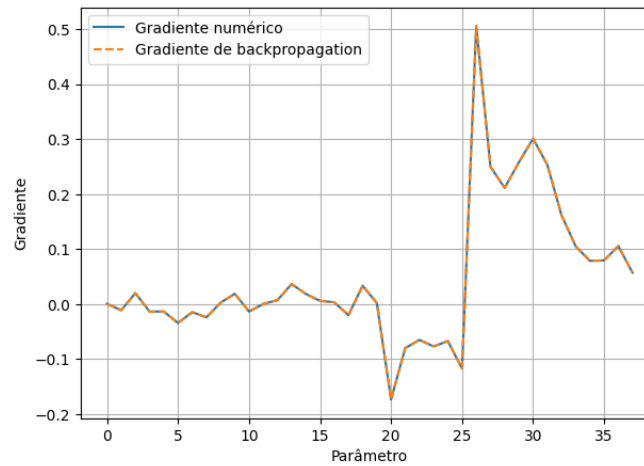


Figura 3: Comparação entre o gradiente de *backpropagation* e o gradiente numérico (diferenças finitas centrais, $\epsilon = 10^{-4}$) na rede miniatura.

Como as diferenças relativas obtidas ($\sim 10^{-11}$) são muito menores que o limiar de referência 10^{-7} , confirma-se a correção da implementação analítica do *backpropagation*, inclusive do termo de regularização no gradiente.

3.3 Comparação de algoritmos de otimização (Parte I)

A Tabela 3 sintetiza o treinamento da rede completa ($[400, 25, 10]$, 5 000 exemplos) com gradiente conjugado, variando λ e o limite de iterações.

Tabela 3: Desempenho do treinamento por Gradiente Conjugado (CG) em função de λ e do limite de iterações. Acurácia medida sobre o próprio conjunto de treinamento (5 000 exemplos).

λ	maxiter	Iter. executadas	Custo final	Acurácia (treino)	Convergiu?
0	100	100	0,044238	99,78%	Não
0	400	246	0,000520	100,00%	Sim
1	100	100	0,353668	98,38%	Não
1	400	400	0,312151	99,48%	Não
10	100	100	1,001030	94,00%	Não
10	400	400	0,986729	94,22%	Não

O gradiente conjugado convergiu de forma muito mais eficiente que o gradiente descendente simples: com $\lambda = 1$ e 400 iterações, o CG atinge 99,48% de acurácia de treino, contra 93,74% do gradiente descendente com 500 iterações. Sem regularização ($\lambda = 0$, 400 iterações), o CG atinge 100% de acurácia, memorizando integralmente o conjunto de treino, o que indica um indício de sobreajuste, já que nesta etapa da Parte I todo o conjunto é usado tanto para treino quanto para avaliação. É importante notar que, exceto para $\lambda = 0$, o critério de parada por tolerância do gradiente (**gtol**) não foi atingido dentro do limite de iterações: o algoritmo foi interrompido pelo **maxiter**, e poderia, a princípio, continuar reduzindo o custo com mais iterações.

3.4 Visualização das unidades escondidas

Descartando a coluna de *bias* de $\Theta^{(1)} \in \mathbb{R}^{25 \times 401}$, cada um dos 25 neurônios escondidos fornece um vetor de 400 pesos, reorganizado em imagem 20×20 e normalizado individualmente ($w_{\text{norm}} = (w - w_{\text{min}})/(w_{\text{max}} - w_{\text{min}})$). A Figura 4 mostra o resultado (pesos da rede treinada por CG, $\lambda = 1$, 400 iterações).



Figura 4: Representação visual dos pesos de cada uma das 25 unidades da camada escondida, reformatados em imagens 20×20 e normalizados em $[0, 1]$.

Diferentemente de uma rede convolucional, cujos filtros tendem a apresentar padrões localizados, cada unidade escondida aqui está conectada a todos os pixels da imagem, de modo que os padrões visualizados são mais difusos; ainda assim, é possível observar regiões de maior contraste que lembram traços típicos de determinados dígitos, sugerindo que cada unidade funciona como um detector parcial de traços, cuja combinação na camada de saída produz a classificação final.

3.5 Curva de aprendizado (Parte II)

A Figura 5 relaciona os custos sem regularização de treino e validação com o tamanho do conjunto de treinamento, para $\lambda = 1$ fixo.

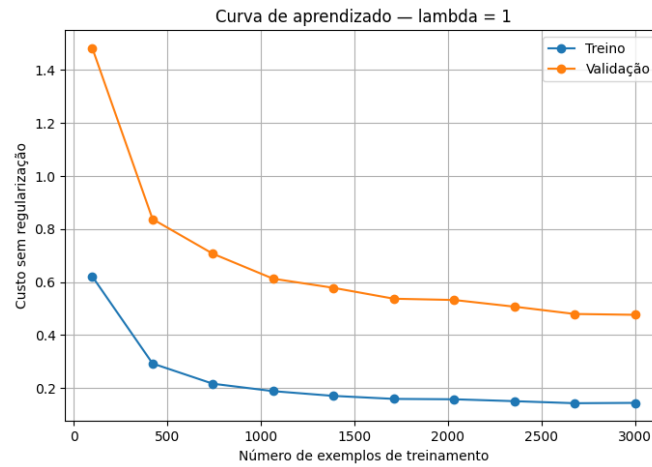


Figura 5: Curva de aprendizado: custo sem regularização de treino e validação em função do número de exemplos de treinamento m ($\lambda = 1$).

Para $m = 100$, o custo de treino é baixo (0,621301) e o de validação é elevado (1,482846), evidenciando alta variância. Conforme m cresce até 3000, o custo de validação cai continuamente até 0,476619, aproximando-se do custo de treino final (0,144472), embora um *gap* residual permaneça. Isso mostra que aumentar a quantidade de dados de treino reduz a variância e melhora a generalização, mas sugere também que ainda há espaço para redução adicional do erro (por exemplo, com mais dados ou maior capacidade do modelo). Como λ é mantido fixo enquanto m cresce, a razão λ/m diminui ao longo da curva, reduzindo o peso relativo da regularização.

3.6 Seleção automática do parâmetro de regularização

A Tabela 4 e a Figura 6 apresentam o custo de treino e de validação (sem regularização) para os 10 valores de λ testados, treinando a rede apenas com o conjunto de treino.

Tabela 4: Avaliação do impacto de λ nos custos de treino e validação.

λ	Custo treino	Custo validação	Diagnóstico
0	0,000412	1,069702	Sobreajuste severo (variância alta)
0,001	0,000558	1,015357	Sobreajuste
0,003	0,000879	0,974482	Sobreajuste
0,01	0,002521	0,725584	Sobreajuste moderado
0,03	0,006526	0,661576	Redução de variância
0,1	0,018395	0,578056	Transição
0,3	0,050683	0,506196	Bom ajuste
1	0,144472	0,476619	Ponto ótimo (λ^* selecionado)
3	0,333112	0,510962	Início de subajuste
10	0,680281	0,759881	Subajuste severo (viés alto)

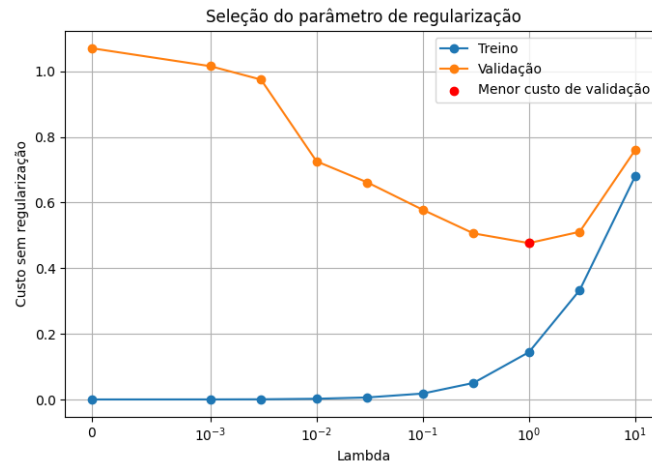


Figura 6: Custo de treino e de validação em função de λ (escala simétrica-log no eixo horizontal). O ponto vermelho indica o menor custo de validação.

A tabela evidencia com clareza o compromisso entre viés e variância: para $\lambda \rightarrow 0$, a penalização é praticamente nula e a rede memoriza o treino ($J_{\text{treino}} \approx 0$), mas generaliza mal ($J_{\text{val}} = 1,069702$); para $\lambda = 10$, a penalização excessiva eleva tanto J_{treino} quanto J_{val} (subajuste). O mínimo do custo de validação ocorre em $\lambda^* = 1$, que foi selecionado como parâmetro de regularização ótimo.

3.7 Avaliação final no conjunto de teste

Com $\lambda^* = 1$ selecionado, o modelo foi avaliado nos três conjuntos (Tabela 5). Como destacado na Seção 2.8, o conjunto de teste foi mantido isolado durante todo o processo de treinamento e de escolha de λ , sendo utilizado exclusivamente nesta etapa final; por isso, a acurácia de teste obtida constitui uma estimativa não-viesada do desempenho de generalização do modelo.

Tabela 5: Métricas finais do modelo selecionado ($\lambda^* = 1$, 400 iterações de CG).

Conjunto	Amostras	Custo (sem reg.)	Erro	Acurácia
Treino	3 000	0,144472	0,33%	99,67%
Validação	1 000	0,476619	7,00%	93,00%
Teste	1 000	0,477752	6,30%	93,70%

A rede classificou corretamente 937 das 1000 imagens de teste, com 63 erros. A proximidade entre o custo de validação (0,476619) e o de teste (0,477752) é consistente com um processo sem vazamento de dados entre os conjuntos. A Figura 7 mostra uma amostra dos erros de classificação no teste.

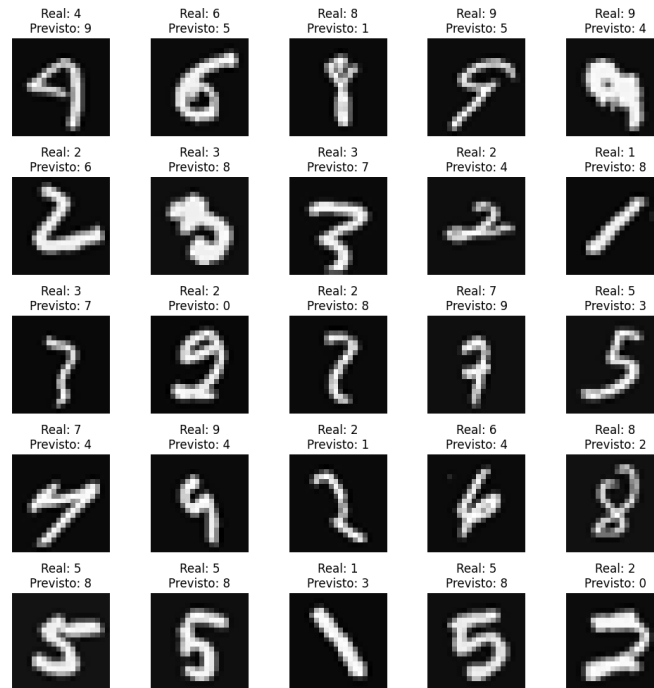


Figura 7: Exemplos de imagens do conjunto de teste classificadas incorretamente, com o dígito verdadeiro (Real) e o previsto.

Na visualização dessa amostra de erros, destacam-se confusões entre dígitos 4 e 9 e entre 3 e 8 (ambos os sentidos), tipicamente associadas a traços abertos ou mal definidos nessas regiões da caligrafia; outras confusões pontuais (por exemplo $1 \rightarrow 8$, $2 \rightarrow 6$, $7 \rightarrow 4$) também aparecem na amostra, mas em menor número. Como apenas 25 dos 63 erros totais do conjunto de teste são exibidos na Figura 7, essa análise qualitativa se restringe aos casos efetivamente observados nessa amostra.

4 Conclusão

Neste trabalho, implementou-se com sucesso um Perceptron Multicamadas regularizado com uma camada escondida para o reconhecimento de dígitos manuscritos da base MNIST, cumprindo os objetivos propostos nas Partes I e II do enunciado.

O algoritmo de *backpropagation* teve sua correção confirmada pela checagem numérica do gradiente, que registrou diferença relativa da ordem de 10^{-11} , muito abaixo do limiar de referência de 10^{-7} . O otimizador de gradiente conjugado (CG) apresentou convergência bem mais rápida que o gradiente descendente convencional, atingindo 99,48% de acurácia de treino com $\lambda = 1$ e 400 iterações, contra 93,74% do gradiente descendente com 500 iterações e $\alpha = 1$.

A visualização dos pesos da camada escondida indicou que as unidades funcionam como detectores parciais e difusos de traços, e não como memorizadores de dígitos completos. A divisão estratificada dos dados e a curva de aprendizado mostraram como o aumento do conjunto de treino reduz a variância do modelo, e a busca automatizada do parâmetro de regularização identificou $\lambda^* = 1$ como o valor que minimiza o custo de validação, evidenciando claramente os regimes de sobreajuste (λ pequeno) e de subajuste (λ grande). Ao longo de toda a Parte II, o conjunto de teste foi mantido fora tanto do treinamento quanto da escolha de λ , sendo usado apenas na avaliação final; essa separação rigorosa é o que garante que a acurácia de teste reportada seja uma medida confiável e não-viesada da capacidade de generalização do modelo.

O modelo final atingiu 99,67% de acurácia no treino, 93,00% na validação e **93,70%** no conjunto de teste independente, um desempenho consistente e satisfatório para uma rede rasa (uma única camada escondida) treinada sobre um conjunto relativamente pequeno de 5 000 exemplos.

Referências

- [1] FLORINDO, J. B. *Material Didático da Disciplina MS571/MT571 — Aprendizado de Máquina*. Instituto de Matemática, Estatística e Computação Científica (IMECC), Universidade Estadual de Campinas (UNICAMP), 2026.
- [2] LECUN, Y.; BOTTOU, L.; BENGIO, Y.; HAFFNER, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, v. 86, n. 11, p. 2278–2324, 1998.
- [3] KAGGLE. *MNIST Dataset*. Disponível em: <https://www.kaggle.com/datasets/hojjatk/mnist-dataset>. Acesso em: 2026.
- [4] MITCHELL, T. M. *Machine Learning*. New York: McGraw-Hill, 1997.
- [5] GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. Cambridge, MA: MIT Press, 2016.
- [6] BISHOP, C. M. *Pattern Recognition and Machine Learning*. New York: Springer, 2006.
- [7] VIRTANEN, P. et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, v. 17, p. 261–272, 2020. Documentação de `scipy.optimize.minimize`: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize.html>.